

M11 - Storage & File Formats Foundations

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

Concept before command

Every lesson explains the mental model and correctness/performance evidence before asking you to run the project.

Contents

- DE11-01** - Row vs columnar storage; where Parquet fits
- DE11-02** - Parquet schema, row groups and metadata
- DE11-03** - Projection, predicates and analytical reads
- DE11-04** - Compression, small files and compaction
- DE11-05** - Partitioning strategy and over-partitioning
- DE11-06** - Lake vs warehouse vs lakehouse
- DE11-07** - Delta tables, transaction log and ACID concepts
- DE11-08** - MERGE/upsert and current-state records
- DE11-09** - Schema enforcement, controlled evolution and compatibility
- DE11-10** - Time travel, retention, VACUUM safety and medallion checkpoint

DE11-01 - Row vs columnar storage; where Parquet fits

What you will be able to do

Explain row-oriented vs columnar storage and choose Parquet for analytical scans rather than because it is fashionable.

Why this matters

Storage layout changes how much data an analytical query must read.

Picture it this way

A row store keeps each customer form together; a column store files all amounts together, all regions together, etc., which helps queries that scan a few columns across many rows.

Start from zero

- CSV is text/row-like and convenient for exchange/debugging but weakly typed.
- Parquet is a typed columnar file format designed for analytical workloads.
- Columnar layout helps projection/compression because similar values are stored together.
- Row-oriented databases are often strong for point reads/writes/transactions; Parquet is not a transactional database by itself.
- Choose format based on read/write pattern, interoperability, schema and ecosystem.
- Keep raw source if conversion rules may need replay.

Teacher demonstration

```
python parquet_lab/01_csv_to_parquet.py
# Compare source/orders.csv and produced Parquet:
# rows, columns, schema/types and file sizes.
```

Read the example like English

- The conversion should preserve business rows/values while changing physical representation.
- Parquet carries typed schema metadata.
- A size difference is interesting but not universal performance proof.

Expected check

format	strength
CSV	human-readable exchange
Parquet	columnar typed analytics
database row store	transactional point reads/writes

Common beginner traps

- Calling Parquet a database.
- Assuming columnar is faster for every access pattern.
- Deleting the only raw CSV after first conversion.

Now you do a similar task

Convert the supplied orders CSV to Parquet and explain two queries where columnar layout is helpful and one workload where it may not be the best storage boundary.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why does selecting only two columns matter more for Parquet than for a plain CSV scan?

Answer in your own words before continuing.

DE11-02 - Parquet schema, row groups and metadata

What you will be able to do

Inspect Parquet schema, row groups, column metadata and row counts as part of validation.

Why this matters

Parquet performance/correctness lives partly in metadata. You should be able to inspect what was written without guessing.

Picture it this way

A Parquet file is a set of labeled crates (row groups); inside each crate, each column has its own packed section and summary labels.

Start from zero

- Schema stores field names/types.
- Row groups divide rows into independently described chunks; columns within them have metadata/statistics/compression.
- Metadata can include min/max/null counts depending writer/column, enabling data skipping in readers.
- Row-group sizing influences parallelism/skipping but has no single magic value.
- Validate row count/schema after writing.
- Metadata inspection does not prove every business rule; pair it with data-level checks.

Teacher demonstration

```
import pyarrow.parquet as pq
pf = pq.ParquetFile("output/orders.parquet")
print(pf.schema)
print("row_groups", pf.num_row_groups)
print("rows", pf.metadata.num_rows)
print(pf.metadata.row_group(0))
```

Read the example like English

- ParquetFile reads metadata without first materializing all rows into pandas.
- num_rows should reconcile with source conversion population.
- Schema/types should match your contract.

Expected check

check	proof
schema	expected typed fields
row_groups	record observed count
rows	matches source population
metadata	compression/statistics inspectable

Common beginner traps

- Never inspecting output after write.
- Treating row-group count as a quality score.
- Assuming metadata min/max is always present/useful for every type/writer.

Now you do a similar task

Run the metadata lab and save schema, row count, row-group count and one column's compression/statistics. Explain what each can and cannot prove.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

How can Parquet metadata help a query avoid reading some data?

Answer in your own words before continuing.

DE11-03 - Projection, predicates and analytical reads

What you will be able to do

Use projection and filters so analytical readers can avoid unnecessary columns/data, and measure the logical difference.

Why this matters

Reading all columns/rows wastes I/O and memory when a query needs a small subset.

Picture it this way

Instead of bringing the entire filing cabinet to your desk, request only the amount and region folders and only drawers matching your date/filter.

Start from zero

- Projection means selecting only required columns.
- Predicate filtering reduces rows; Parquet-aware engines may push predicates/use statistics to skip chunks.
- Predicate pushdown capability depends on engine/expression/file metadata; do not promise skipping without evidence.
- Partition pruning and row-group skipping are different mechanisms.
- Measure columns/rows/read plan or timing on representative data; tiny files may show no speed advantage.
- Correctness first: filtered result must equal an equivalent full-read/filter result.

Teacher demonstration

```
import pandas as pd
cols = ["order_id", "region", "qty", "unit_price"]
df = pd.read_parquet("output/orders.parquet", columns=cols)
west = df[df["region"] == "West"]
print(len(west), west.columns.tolist())
```

Read the example like English

- The read requests four columns rather than all source columns.
- Filter is applied after read in this simple pandas example; other engines/APIs may push predicates.
- Validate the filtered rows against source truth.

Expected check

concept	meaning
projection	fewer columns
predicate	fewer rows
partition pruning	skip directory partitions
row-group skipping	metadata-based chunk skip

Common beginner traps

- Saying pandas filter necessarily pushed into Parquet scan.
- Projecting away a key needed for validation.
- Benchmarking a 1 KB file and generalizing to production.

Now you do a similar task

Use the supplied projection/filter lab. Read a narrow column set, filter a region/status, and prove the result matches a full-read equivalent.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What is the difference between projection and predicate filtering?

Answer in your own words before continuing.

DE11-04 - Compression, small files and compaction

What you will be able to do

Explain compression and the small-files problem, then compact many tiny analytical files without changing row/control totals.

Why this matters

Thousands of tiny files create metadata/list/open overhead and poor scan efficiency even when total bytes are modest.

Picture it this way

It is faster to move a few well-packed boxes than thousands of envelopes containing one line each.

Start from zero

- Parquet applies encodings/compression per column/page; similar column values often compress well.
- Compression trades CPU for reduced I/O/storage; codec choice depends ecosystem/workload.
- Small-files problem comes from file listing/open/task overhead and poor row-group utilization.
- Compaction reads multiple small files and writes fewer larger files while preserving data.
- Compaction is a new physical layout operation; validate row/key/measure totals before deleting/retiring old files.
- Do not chase one ideal file size on a tiny laptop lab; learn the reasoning/evidence.

Teacher demonstration

```
python small_files/01_create_small_files.py
python small_files/02_compact_small_files.py
# Compare file count before/after and validate row totals.
```

Read the example like English

- First script deliberately creates inefficient small output files.
- Second rewrites them into fewer files.
- Business data must reconcile before/after compaction.

Expected check

proof	expected
file count	decreases
row count	unchanged
key set/measure totals	unchanged
storage layout	new files

Common beginner traps

- Deleting small files before validating compacted output.
- Calling gzip/codec choice the same thing as compaction.
- Over-compacting into a single giant file regardless of parallelism.

Now you do a similar task

Run small-file creation/compaction and save file counts + row/control totals before and after. Explain why fewer files can help even if total bytes are similar.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What pipeline correctness risk exists if compaction is treated as "just a storage optimization" with no reconciliation?

Answer in your own words before continuing.

DE11-05 - Partitioning strategy and over-partitioning

What you will be able to do

Choose partition columns with filter selectivity/cardinality/file-size reasoning and recognize over-partitioning.

Why this matters

Partition directories can skip large groups quickly, but high-cardinality partitions create tiny files/directories and metadata overhead.

Picture it this way

Partitioning is shelving books by a useful category. Shelving by unique order ID creates one shelf per book and defeats the purpose.

Start from zero

- Partitioning physically groups files by values such as date/region.
- Best candidates match common filters and have manageable cardinality/data volume.
- Partition pruning skips directories without opening their files when filter matches partition columns.
- High-cardinality keys (order_id/customer_id) are usually poor coarse partitions.
- Multiple partition columns multiply directory combinations.
- Validate rows and file distribution per partition; watch skew/empty/tiny partitions.

Teacher demonstration

```
# Example writer idea
# df.to_parquet("out/orders", partition_cols=["order_date"])
# Good only if date is commonly filtered and each date has enough data.
# Avoid partition_cols=["order_id"] for unique IDs.
```

Read the example like English

- Date partitioning can make a one-day query scan only that date directory.
- Unique order_id would create near-one-row partitions/files.
- The correct choice depends on volume/query patterns.

Expected check

candidate	reasoning
event_date	often useful bounded cardinality/filter
region	possible if few balanced values
order_id	usually over-partitioned
status	can be skewed/low benefit

Common beginner traps

- Partitioning every field used in WHERE.
- Using unique keys as folder partitions.
- Assuming partitioning itself sorts data within files.

Now you do a similar task

Write a partition plan for the supplied orders dataset at 10 million rows/year. Compare date, region and order_id on cardinality, common filters and expected file distribution.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why can partitioning by order_id make a data lake slower rather than faster?

Answer in your own words before continuing.

DE11-06 - Lake vs warehouse vs lakehouse

What you will be able to do

Distinguish data lake, warehouse and lakehouse by storage/table/governance/query capabilities rather than slogans.

Why this matters

Architecture choices affect cost, open formats, transactions, BI performance and governance.

Picture it this way

A lake is a flexible raw storage yard; a warehouse is a managed structured store; a lakehouse tries to add warehouse-like table management to lake/object files.

Start from zero

- A data lake stores files/object data flexibly, often open formats and multiple data types.
- A warehouse offers managed structured tables, SQL optimization/concurrency/governance.
- A lakehouse uses lake/object storage plus table metadata/transaction semantics (e.g., Delta/Iceberg/Hudi class of technologies) to improve reliability/query behavior.
- These categories overlap by cloud product; decide from required capabilities, not marketing labels.
- Parquet files alone do not make a lakehouse.
- A lakehouse still needs quality/modeling/governance; ACID table metadata does not make bad data clean.

Teacher demonstration

```
Plain Parquet files
+ no table log/transaction protocol
= file dataset
Parquet + Delta transaction log/table protocol
= managed lakehouse-style table semantics
```

Read the example like English

- The file format and table format are separate concepts.
- Warehouse/lakehouse decisions depend on workloads, governance and operations.

Expected check

system	typical strength
lake	flexible file/object storage
warehouse	managed SQL analytics/concurrency
lakehouse	open-ish lake files + table transactions/metadata

Common beginner traps

- Saying lakehouse replaces every warehouse.
- Calling a Parquet folder a Delta table.
- Ignoring operational/governance requirements.

Now you do a similar task

Given three workloads (raw log archive, high-concurrency BI, Spark/Python analytics with upserts), choose lake/warehouse/lakehouse tendencies and defend tradeoffs.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What extra capability turns a folder of Parquet files into something more table-like?

Answer in your own words before continuing.

DE11-07 - Delta tables, transaction log and ACID concepts

What you will be able to do

Understand Delta table files plus transaction log as a table protocol providing atomic versions/ACID-style operations over files.

Why this matters

Plain file rewrites can leave readers seeing partial or inconsistent sets. A transaction log records which files form each committed table version.

Picture it this way

The data files are boxes in a warehouse; the Delta log is the official inventory ledger saying which boxes belong to version 7 of the table.

Start from zero

- Delta table commonly stores Parquet data files plus **_delta_log** transaction log.
- Writers commit actions/version metadata so readers resolve a consistent snapshot.
- Atomic commits prevent readers from treating half a multi-file update as a completed table version.
- ACID here describes table transaction semantics; external source-to-table end-to-end exactly-once still needs pipeline design.
- History/version metadata enables time travel and audit.
- Do not manually edit/delete files in a Delta table directory because the log is authoritative.

Teacher demonstration

```
python delta_lab/01_create_delta.py
python delta_lab/02_inspect_transaction_log.py
# Inspect table folder: Parquet files + _delta_log entries.
```

Read the example like English

- Creation should produce **_delta_log**.
- Log inspection shows committed actions/version information.
- Readers use the log to know the active file set.

Expected check

object	role
_delta_log	transaction/version metadata
Parquet files	physical data
table version	consistent snapshot
manual file deletion	unsafe

Common beginner traps

- Deleting "old" Parquet files manually.
- Calling Delta exactly-once for your whole pipeline automatically.
- Ignoring transaction history during debugging.

Now you do a similar task

Create the supplied Delta table, inspect the first log/version and explain how a reader knows which data files are active.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why is the transaction log more important than simply listing every Parquet file in the folder?

Answer in your own words before continuing.

DE11-08 - MERGE/upsert and current-state records

What you will be able to do

Use MERGE/upsert to apply new/changed keys to a current-state Delta table with replay-safe logic.

Why this matters

Incremental sources contain inserts and updates. Blind append creates multiple current rows for one key.

Picture it this way

MERGE is a mailroom rule: if the customer already has a folder, update it; otherwise create a new folder.

Start from zero

- MERGE matches source rows to target by a business key.
- whenMatched... applies update rule; whenNotMatched... inserts new keys.
- Source must be deduplicated to one intended row per target key for deterministic semantics.
- Use source version/updated_at rules so older events do not overwrite newer state.
- Rerun the same source batch and prove target key count/state is stable.
- MERGE creates new Delta table versions; validate keys/control totals after commit.

Teacher demonstration

```
# Conceptual Delta merge
(target.alias("t")
 .merge(source.alias("s"), "t.order_id = s.order_id")
 .when_matched_update_all()
 .when_not_matched_insert_all()
 .execute())
```

Read the example like English

- Join condition defines current-state business identity.
- Matched update vs not-matched insert create upsert behavior.
- Without a source winner rule, duplicate source keys can be ambiguous/dangerous.

Expected check

test	expected
new key	inserted
existing newer update	updated
same batch replay	no duplicate key
older update	does not overwrite if policy enforces version

Common beginner traps

- MERGE on non-unique/incorrect key.
- Using update_all when source schema/semantics differ.
- Never testing replay.

Now you do a similar task

Run the MERGE lab and record target row count/key state before, after and after exact replay. Explain the source-key policy.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why should you deduplicate/version-filter the source before MERGE?

Answer in your own words before continuing.

DE11-09 - Schema enforcement, controlled evolution and compatibility

What you will be able to do

Reject incompatible writes while allowing controlled schema evolution only when meaning/compatibility is reviewed.

Why this matters

Schema evolution convenience can silently create inconsistent columns/types if every change is auto-accepted.

Picture it this way

A table contract is a form. Adding an approved optional field is manageable; changing "amount" from number to arbitrary text is not a harmless extra field.

Start from zero

- Schema enforcement rejects writes that do not match the table contract.
- Schema evolution may add compatible fields through explicit option/DDL depending engine/library.
- Not every type change is safe; semantic compatibility matters beyond physical type.
- Required downstream consumers/tests should be reviewed before evolving.
- Record schema version/change in lineage and test old/new readers.
- Quarantine incompatible input rather than weakening the table blindly.

Teacher demonstration

```
python delta_lab/04_schema_enforcement_evolution.py
# Observe a rejected incompatible write, then the controlled approved evolution path.
```

Read the example like English

- The exercise separates "write failed because contract protects table" from "approved schema evolution".
- A successful mergeSchema/evolution is not evidence every downstream consumer understands the new field.

Expected check

change	decision
new optional field	possibly controlled evolution
remove required field	breaking
numeric -> free text	breaking
rename without compatibility plan	breaking

Common beginner traps

- Enabling automatic schema evolution globally.
- Changing table schema to make a bad source batch pass.
- No downstream compatibility test.

Now you do a similar task

Run the schema lab, save one enforcement failure and one approved evolution. Explain why each decision is correct.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why is "the engine can add this column" not the same as "the data contract should add this column"?

Answer in your own words before continuing.

DE11-10 - Time travel, retention, VACUUM safety and medallion checkpoint

What you will be able to do

Use version history/time travel safely, understand retention/VACUUM tradeoffs and connect storage design to bronze-silver-gold layers.

Why this matters

Historical versions help debugging/recovery, but physical old files are not guaranteed forever. Maintenance can remove files needed by old versions.

Picture it this way

Time travel is viewing earlier ledger snapshots; VACUUM is throwing away unneeded old boxes. Once discarded, some old snapshots cannot be reconstructed.

Start from zero

- Time travel reads a previous committed table version/timestamp while required data files still exist.
- Retention determines how long old files/history remain available for recovery/debugging according to system settings/policy.
- VACUUM removes files no longer referenced by recent table state after safety rules; aggressive retention can break old time travel/recovery/concurrent readers.
- Never disable retention safety/copy destructive VACUUM commands in a beginner lab without understanding implications.
- Bronze typically preserves raw/source-like data, Silver validates/standardizes, Gold serves modeled aggregates/products.
- Medallion layers are contracts, not merely folder colors; every layer needs grain/quality/rebuild rules.

Teacher demonstration

```
python delta_lab/05_time_travel.py
python 01_medallion_build.py
# Read delta_lab/VACUUM_SAFETY_v1.0.txt before any maintenance experiment.
```

Read the example like English

- Time-travel lab demonstrates reading earlier table state.
- Medallion build validates layer transitions.
- Safety document exists because VACUUM can make old physical files unavailable.

Expected check

concept	proof
history	versions listed
time travel	older version read
vacuum	not run destructively without retention plan
bronze/silver/gold	contracts + reconciliation

Common beginner traps

- VACUUM 0 HOURS copied from internet on important table.
- Calling Bronze raw but transforming/deduping away source evidence.
- Assuming history metadata guarantees old data files remain forever.

Now you do a similar task

Run time-travel + medallion labs, record version/control totals, and write a safe retention/VACUUM policy for a production-like table.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why can a Delta version still exist in history while time travel to it eventually becomes impossible?

Answer in your own words before continuing.